

SADA: Semantic Augmentation for Focus-Oriented Data Analysis

Supplementary Appendix

Zhuo Wang, Xintao Lin et al.

Contents

1	Trust and Provenance	1
2	Schema-Structure Validity and Artifacts	2
3	Execution Details: Cost Model and Sub-agent Isolation	2
4	LLM Judge Prompts for Experiments 1–4	3
4.1	Experiment 1: Pairwise Report Grading	3
4.2	Experiment 2: Analytical-Form Classification	4
4.3	Experiment 3: Characteristic Adherence	5
4.4	Experiment 4: Row-Level Grounding Audit	9

1 Trust and Provenance

A light, optional layer built on the operator’s content-addressed artifacts; not claimed as an independent contribution. The reader may view it as an extension of the engine’s “schema integrity as provenance” (Section 4 of the main paper) onto the deliverable. Every stage’s inputs and outputs are registered into `artifact_manifest.json` by sha256, and each LLM stage additionally persists a trace, so an entire run can be independently audited, reproduced, and traced from disk artifacts alone. On this basis it offers two analyst-facing mechanisms.

Row-level evidence column. Each augmented column may be paired with an evidence column recording the source columns it was derived from. The evidence is *emitted together* within the augmentation extraction call rather than computed separately, so the added cost is negligible.

Low-confidence row surfacing. Text analysis has inherently ambiguous cases (“OK, thanks” could be positive or neutral); the operator’s extraction contract requires it to *abstain* (emit `null/Unknown`) when evidence is insufficient, and to surface those few rows — with row index, original text, and the uncertain value — to the analyst for spot-checking, with the persisted sub-agent logs and trace artifacts available for deeper inspection.

2 Schema-Structure Validity and Artifacts

The delivered table is always flat, but SADA retains the relational structure behind it (a flat feature set, a causal graph, or a concept tree) for provenance, and enforces that this structure is fully reconstructible from JSON artifacts. The root points to existing nodes, each node has a stable `Id`, and every parent/child/edge/selection reference cites a legal node; each materialized category/spec carries `SelectedNodeId`, `Role`, `Parent`, and `StructurePath`, so that every flat column traces back to a node in the schema and to its role in the original analytical intent. If the structure exhibits dangling nodes, a broken selection chain, an incompatible `IntentClass/StructureType`, or a spec that cannot be mapped back to a selected node, the executor treats that stage’s output as invalid and retries or falls back to a flat structure, rather than emitting an untraceable `specs.json`.

3 Execution Details: Cost Model and Sub-agent Isolation

This section records the execution-related engineering choices behind the engine of Section 4: the planning-time cost model that sets tier and concurrency before a run (function `_estimate_cost`), and the sub-agent isolation under which every stage executes. These are generic engineering choices rather than SADA-specific contributions, and none of them is an experimental claim; we document them for reproducibility.

Token split. Before a run the operator predicts the total number of LLM calls N from the data shape (rows, labels, facets) and the chosen chunk sizes, then estimates per-call input tokens. A cacheable prefix of the prompt is discounted by an assumed cache-hit rate h : cached tokens are $T_{\text{cached}} = N \cdot p_{\text{prefix}} \cdot h$, and the remainder $T_{\text{bill}} = \max(0, T_{\text{in}} - T_{\text{cached}})$ is billed at the uncached rate. The dollar estimate then combines T_{bill} , T_{cached} , and the output tokens at their respective per-million-token prices.

Wall-clock and retries. Wall-clock is approximated as $\lceil N/k \rceil \cdot t_{\text{wave}}$ for concurrency k , where t_{wave} is either a per-model calibration measurement or a default (about 137s for Claude-tier models and 34s otherwise). Retry overhead enters through an expected full-chunk retry rate (default 0.03). Chunk sizes, facets-per-call, and concurrency k are selected by the host model within the recipe’s declared bounds after reading the data profile, and the categorize and tagging stages reuse the same worker budget sequentially rather than stacking into a higher peak. The estimate is a planning-time approximator that assumes roughly equal-length waves; the real t_{wave} fluctuates with wide-text chunks, adaptive shrinking, and retries, so the model is used to choose a configuration beforehand rather than to explain performance afterward.

Sub-agent isolation. Sub-agents perform independent extraction in clean context: each stage is invoked with an isolated prompt, receives only the stage contract and structured input payload, and returns strict JSON that is persisted to stage logs and workdir artifacts. The main agent therefore orchestrates stage boundaries rather than inheriting the sub-agent’s scratchpad, which avoids re-billing the main agent’s long history on every extraction and contains failures within a single chunk or stage. The persisted logs and artifacts also support the auditing of Appendix 1.

4 LLM Judge Prompts for Experiments 1–4

This section reproduces verbatim the prompts given to the fixed LLM judges of Experiments 1–4 in the main paper. All four judges run on `claude-opus-4.8-xhigh`, held constant across every condition and substrate, and every judge is required to emit strict JSON so that scoring is deterministic downstream. Placeholders in braces (`{goal}`, `{report}`, ...) are substituted at call time; reports are stripped of their YAML header and truncated to 9,000 characters per side (Experiment 1) or 12,000 characters (Experiment 2).

4.1 Experiment 1: Pairwise Report Grading

Each unordered pair of reports is judged in both presentation orders with the prompt below; the two orders must agree for a winner to stand (swap stabilization), after which per-unit outcomes feed the Bradley–Terry fit.

```
You are comparing two analyses (A and B) of the SAME dataset
answering the SAME goal. Decide which is BETTER and score each side along
5 fixed dimensions.

Dimensions (score each from 1 to 5; 1=poor, 5=excellent):
- specificity : concrete entities, magnitudes, time windows, dataset rows
- evidence    : claims tied to numbers / tables / column references
- depth       : segmentation, comparison across groups, causal reasoning
                (NOT just description)
- actionability : conclusions are specific to this dataset and decision-ready
- coherence    : internal consistency, no contradictions

Avoid superficial criteria like "longer is better" or "more sections is
better". Score A and B independently on each dimension before deciding
the overall winner. Do not use external ground truth.

Augmented-column citation is NOT a sixth quality dimension. It is an audit
field: cite a listed augmented column only when the report explicitly names
that column or a directly equivalent derived field.

Evidence references must be short quotes or table/column references visible
inside the reports. Prefer numeric evidence. If a side has no usable evidence
for a dimension, return an empty list for that side.

== ANALYTICAL GOAL ==
{goal}

== AUGMENTED COLUMNS FOR A ==
{aug_cols_a}

== AUGMENTED COLUMNS FOR B ==
{aug_cols_b}

== ANALYSIS A ==
{report_a}

== ANALYSIS B ==
{report_b}

== END ==

Return STRICT JSON ONLY:

{
  "scores_a": {"specificity": 1-5, "evidence": 1-5, "depth": 1-5,
               "actionability": 1-5, "coherence": 1-5},
  "scores_b": {"specificity": 1-5, "evidence": 1-5, "depth": 1-5,
               "actionability": 1-5, "coherence": 1-5},
  "winner": "A" | "B" | "TIE",
```

```

"margin": "small" | "medium" | "large",
"confidence": <number 0.0-1.0>,
"evidence_refs": {
  "A": [{"dimension": "specificity|evidence|depth|actionability|coherence",
    "quote": "short quote", "why": "short reason"}],
  "B": [{"dimension": "specificity|evidence|depth|actionability|coherence",
    "quote": "short quote", "why": "short reason"}]
},
"augmented_column_refs": {
  "A": [{"column": "column name", "quote": "short quote",
    "dependent": true | false}],
  "B": [{"column": "column name", "quote": "short quote",
    "dependent": true | false}]
},
"reason": "one short sentence"
}

```

Use TIE only when the two are genuinely indistinguishable in quality.
The winner / margin should be consistent with the dimension scores but
you may weight dimensions by your own judgment.
Output ONLY the JSON.

4.2 Experiment 2: Analytical-Form Classification

A single classifier call per finished report assigns it to exactly one of the three mutually exclusive forms C1/C2/C3 of §5.3 of the main paper.

You are classifying the ANALYTICAL FORM of a data-analysis report. The report analyzes a table that has both TEXT columns and NUMERIC columns, answering an analytical goal.

Classify the report into EXACTLY ONE of three categories by HOW it treats text vs numeric signal:

- C1_all_qualitative: the report is essentially qualitative; little or no quantitative computation (few/no numbers, rates, group means, correlations). Conclusions are descriptive.
- C2_text_qual_numeric_quant: the report does QUANTITATIVE analysis on the ORIGINAL numeric columns, but treats the TEXT columns only QUALITATIVELY (mentions/among themes, no structured quantification of text-derived signal).
- C3_text_augmented_quant: the report QUANTIFIES signal DERIVED FROM TEXT (i.e. text is turned into structured/categorical/numeric features and then used in quantitative analysis: group rates, comparisons, model over text-derived columns), AND also quantifies numeric columns.

Key distinction C2 vs C3: does the report compute quantitative results OVER SIGNAL EXTRACTED FROM TEXT (C3), or does it only quantify pre-existing numeric columns while text stays qualitative (C2)?

== ANALYTICAL GOAL ==
{goal}

== REPORT ==
{report}
== END ==

Return STRICT JSON ONLY:
{"category": "C1_all_qualitative" | "C2_text_qual_numeric_quant"
| "C3_text_augmented_quant",
"confidence": 0.0-1.0, "reason": "one short sentence"}

4.3 Experiment 3: Characteristic Adherence

The adherence judge sees only the *added* columns — each with its name and the top-20 counts of its observed values, tallied over every row — together with the analytical query and the *names* of the original columns. The payload also carries a description and declared-domain field per column; both are left empty in the reported runs, so the judge scores column names against observed value distributions. It never sees the original table’s values, the row texts, any column row by row, or a ground-truth schema. The intent sub-type is not inferred by the judge: it is taken from the benchmark’s own annotation for that unit, so the same sub-type is applied to every condition and substrate. The judge emits only checkable component signals; the headline *adherence* number is composed from them outside the model by the fixed per-sub-type rules below, so the judge cannot silently re-weight the technical claim under test.

Adherence composition rules. All six map into $[0, 1]$. `unnamed_bonus` is 1 if at least one identified confounder was *not* named or clearly implied in the query and 0 otherwise; `redundancy` is computed deterministically from the added column names, not judged.

```

predictive_feature_engineering : predictor_fraction * predictive_utility
exploratory_data_analysis      : predictor_fraction * relationship_informativeness
causal_attribution             : treatment_present      * (0.4 + 0.6*unnamed_bonus)
                               * confounder_quality
what_if                        : treatment_intervenable * (0.4 + 0.6*unnamed_bonus)
                               * confounder_quality
faceted_decomposition          : facet_fraction * mece * (1 - redundancy)
focus_inference               : focus_coherence * structure_quality * (1 - redundancy)

where mece = sqrt(mutual_exclusivity * exhaustiveness)
      predictor_fraction / facet_fraction = share of added columns the judge
                                         marks as such

```

Table 1: Causal component signals behind Experiment 3 adherence, averaged over both substrates. Higher is better; bold marks the best variant within each row and component.

Sub-type	Variant	Treatment presence	Confounder presence	Confounder quality
Attribution	skill_off	0.84	0.61	0.23
	skill_on	0.89	0.84	0.41
	skill_on_e2e	1.00	0.84	0.44
What-If	skill_off	0.85	0.35	0.17
	skill_on	0.91	0.85	0.44
	skill_on_e2e	0.97	0.88	0.49

Shared frame. The rubric of the unit’s sub-type is spliced in at `{rubric}`.

```

You are a strict evaluator of a text-to-table augmentation operator, grading
INSTRUCTION FOLLOWING.
An augmentation system was asked to serve the analytical query below and ADDED
the columns listed. Your job is NOT to ask whether the columns are 'reasonable
in general' -- it is to test whether they satisfy the SPECIFIC technical claim
for this intent subtype, stated below. A set of columns can be perfectly
sensible and still FAIL this test.

INTENT SUBTYPE: {subtype_label}

```

Table 2: Additional component signals behind Experiment 3 adherence, averaged over both substrates. Higher is better except leakage; best per sub-type and component is in **bold**.

<i>Correlational</i> : predictive relevance				
Sub-type	Variant	Predictor fraction	Interp./utility	Leakage↓
EDA	skill_off	0.55	0.56	—
	skill_on	0.91	0.96	—
	skill_on_e2e	0.89	0.94	—
PFE	skill_off	0.61	0.47	0.20
	skill_on	0.73	0.59	0.18
	skill_on_e2e	0.71	0.56	0.18

<i>Focus-internal</i> : constitutive facets			
Sub-type	Variant	Facet fraction	MECE
Faceted	skill_off	0.37	0.37
	skill_on	0.61	0.50
	skill_on_e2e	0.65	0.53

{rubric}

Judge the SEMANTICS of each added column. Ignore naming style, casing, and column order, and do not compare against any hidden reference schema. Be conservative: a mechanical or generic set of columns (text length, token counts, bare sentiment) does not realise any of these claims.

Evaluation input JSON:

```
{
  "analytical_query": "...",
  "focus_variable": "...",
  "concept_to_decompose": "...",
  "original_table_columns": [...],
  "added_columns": [
    {
      "name": "...",
      "description": "...",
      "domain": [...],
      "observed_top_values": [...]
    }
  ]
}
```

PFE rubric.

TECHNICAL CLAIM UNDER TEST (predictive feature engineering):

The added columns must be usable FEATURES for PREDICTING the focus variable. Each should be a variable that lies OUTSIDE the focus (not a restatement, not a component or a deterministic function of it, and not something only knowable after the outcome -- that would be leakage) and should plausibly carry predictive signal about it.

Judge each column on: is it a legitimate predictor (predictor), or does it restate/leak the focus (leakage), or is it simply unrelated (irrelevant)?

Return strict JSON only:

```
{
  "subtype": "predictive_feature_engineering",
  "focus_variable": "...",
  "per_column": [
    {
      "name": "...",
      "role": "predictor|leakage|restatement|irrelevant",
      "is_predictor": true,
      "rationale": "..."
    }
  ],
  "n_predictors": 0,
  "predictive_utility": 0.0, // 0-1: as a SET, how much genuine predictive
                           // leverage do these give a model of the focus
                           // (beyond the original columns)?
  "rationale": "..."
}
```

EDA rubric.

TECHNICAL CLAIM UNDER TEST (exploratory data analysis):
The added columns must open up INFORMATIVE RELATIONSHIPS around the focus for a human explorer. Each should be an external, associational variable that can be grouped/segmented on and compared against the focus. Unlike feature engineering, the goal is human UNDERSTANDING, not model input: columns must have interpretable, well-separated categories or scales, not opaque scores. Penalise columns that merely restate the focus, and columns whose values are so fine-grained or so near-constant that no meaningful comparison can be drawn.

Return strict JSON only:

```
{
  "subtype": "exploratory_data_analysis",
  "focus_variable": "...",
  "per_column": [{"name": "...", "role": "predictor|restatement|irrelevant",
                    "is_predictor": true,
                    "is_interpretable_segmentation": true, "rationale": "..."}],
  "n_predictors": 0,
  "relationship_informativeness": 0.0,    // 0-1: as a SET, how well do these
                                          // support discovering interpretable
                                          // relationships with the focus?
  "rationale": "..."}
}
```

Attribution rubric.

TECHNICAL CLAIM UNDER TEST (causal attribution):

The added columns must set up a CAUSAL ESTIMATION around the focus (the OUTCOME). Specifically they must supply:

- (a) a TREATMENT -- the candidate cause whose effect on the outcome is being attributed; and
- (b) CONFOUNDERS -- variables that plausibly cause BOTH the treatment and the outcome, and which therefore MUST be adjusted for or the attribution is biased.

THE DECISIVE TEST: a confounder is only evidence of real causal reasoning if the system SURFACED it from the data/text rather than copying it from the query. For EACH confounder you identify, decide whether that variable was ALREADY NAMED OR CLEARLY IMPLIED in the analytical query text. Set "named_in_query": true if it was. Confounders with "named_in_query": false are the ones that demonstrate the capability under test.
Do NOT count mediators (on the causal path from treatment to outcome) or colliders as confounders -- adjusting for those is a mistake, and labelling them confounders is an error you should mark.

Return strict JSON only:

```
{
  "subtype": "causal_attribution",
  "outcome_variable": "...",
  "per_column": [{"name": "...",
                    "causal_role":
                      "treatment|confounder|mediator|collider|outcome|other",
                    "named_in_query": false, "rationale": "..."}],
  "treatment_present": false, "treatment_columns": [],
  "confounders": [{"name": "...", "named_in_query": false,
                    "plausibility": 0.0}],
  "confounder_quality": 0.0,    // 0-1: are the confounders genuine common
                               // causes of BOTH treatment and outcome (not
                               // mediators/colliders/noise)?
  "rationale": "..."}
}
```

What-If rubric. Identical in structure to Attribution, but the treatment must additionally be intervenable, and `treatment_intervenable` replaces `treatment_present` as the gate in the com-

position rule.

```
TECHNICAL CLAIM UNDER TEST (what-if / intervention):
The added columns must set up an INTERVENTIONAL question about the focus (the
OUTCOME). This is stricter than attribution:
(a) the TREATMENT must be something an actor could actually SET, CHANGE, or
    DECIDE -- an intervenable lever (a policy, an action, an assignment, a
    configurable property). An immutable attribute (e.g. an intrinsic trait,
    a date that already happened) is NOT a valid what-if treatment, even if
    it causes the outcome; and
(b) CONFOUNDERS -- common causes of both the treatment and the outcome, which
    must be adjusted for before the counterfactual effect can be read off.

THE DECISIVE TEST: for EACH confounder, decide whether it was ALREADY NAMED OR
CLEARLY IMPLIED in the query text ("named_in_query"). Confounders NOT named in
the query are the ones that demonstrate the capability under test.
Do NOT count mediators or colliders as confounders.

Return strict JSON only:
{
  "subtype": "what_if",
  "outcome_variable": "...",
  "per_column": [{"name": "...",
                    "causal_role":
                      "treatment|confounder|mediator|collider|outcome|other",
                    "named_in_query": false, "rationale": "..."}],
  "treatment_present": false, "treatment_columns": [],
  "treatment_intervenable": false, // is at least one identified treatment
                                   // actually settable/changeable?
  "confounders": [{"name": "...", "named_in_query": false,
                    "plausibility": 0.0}],
  "confounder_quality": 0.0,
  "rationale": "..."}
}
```

Faceted rubric. The concept to decompose is the focus recorded in the benchmark annotation for that unit, so it is identical across conditions and is never derived from the output being judged.

```
TECHNICAL CLAIM UNDER TEST (faceted decomposition):
The added columns must be constitutive FACETS / PARTS of the focus concept -- a
part-of (meronymy) relationship. A facet is 'an aspect OF' the concept, NOT an
external predictor of it, NOT a cause of it, and NOT a restatement of the whole
concept.

Then judge the facet set on MECE:
* mutual_exclusivity [0-1]: do the facets carve the concept into
    NON-OVERLAPPING aspects? Two columns capturing the same underlying aspect
    under different names lowers this.
* exhaustiveness [0-1]: do the facets together COVER the concept's main
    constitutive aspects? Judge against the concept itself, not against column
    count.

Return strict JSON only:
{
  "subtype": "faceted_decomposition",
  "concept": "...",
  "per_column": [{"name": "...", "is_constitutive_facet": true,
                    "role": "facet|external_predictor|cause|restatement|
                           irrelevant",
                    "rationale": "..."}],
  "facet_columns": [], "non_facet_columns": [],
  "mutual_exclusivity": 0.0,
  "exhaustiveness": 0.0,
  "rationale": "..."}
}
```

Focus-Inf rubric. With no focus named there is deliberately no ground-truth focus, so the judge infers the implied focus from the columns and scores their internal coherence. It also reports `focus_actionability`, which we record but deliberately exclude from the composition rule, since actionability is a property of the proposed focus rather than of the schema built around it.

```
TECHNICAL CLAIM UNDER TEST (focus inference):
The query names NO focus -- the system must first PROPOSE one. By design there
is NO ground-truth focus, so do not anchor on any particular answer. Instead
judge:
  * focus_coherence [0-1]: do the added columns collectively centre on ONE
    coherent, data-inferable focus concept, rather than being a scattered
    grab-bag? Infer the implied focus yourself.
  * structure_quality [0-1]: given that implied focus, do the columns form a
    sound analytical structure around it (coherent facets, or a coherent
    predictor/attribute set) at a useful granularity -- neither a single
    restatement nor undifferentiated noise?
  * focus_actionability [0-1]: is the implied focus one an analyst could
    actually act on / investigate further, as opposed to a trivial or vacuous
    framing?

Return strict JSON only:
{
  "subtype": "focus_inference",
  "implied_focus": "...",
  "per_column": [{"name": "...", "supports_focus": true, "rationale": "..."}],
  "focus_coherence": 0.0,
  "structure_quality": 0.0,
  "focus_actionability": 0.0,
  "rationale": "..."
}
```

4.4 Experiment 4: Row-Level Grounding Audit

The grounding auditor judges each augmented cell against the original data of the row it belongs to. Rows are batched 20 at a time in a single call, each carrying an explicit `row_index`; the response is validated for strict row alignment and column coverage, and any missing row triggers a retry rather than a silent drop. Source cell values are clipped to 8,000 characters.

```
You audit Large Language Model augmented columns of data table, ROW by ROW.
Each augmented value was produced by a model that read the row's original data
(mostly free text plus some structured columns). For EACH augmented
(column,value) on EACH row, decide whether the value is grounded in THAT ROW's
original data:

- SUPPORTED: the value has a direct, literal basis in the row's text/structured
  columns.
- INFERABLE: no literal basis, but the value can be inferred from SPECIFIC
  WORDS/PHRASES in THIS ROW's original text. The inference chain MUST be
  anchored to the row's actual content --- NOT world knowledge, NOT guessing
  beyond the text.
- HALLUCINATED: the value contradicts the row's original data, or the row
  contains nothing that supports it.

Judge each row independently using ONLY that row's original data. Be strict: if
the only support is generic world knowledge not tied to this row's words, it is
HALLUCINATED, not INFERABLE.

You are given a BATCH of rows. For EVERY row_index in the input you MUST return
exactly one entry, with a verdict for EVERY augmented column given for that row.
Output STRICT JSON ONLY:
{"rows": [{"row_index": <int>, "verdicts": [{"column": "<name>",
  "verdict": "SUPPORTED|INFERABLE|HALLUCINATED",
  "evidence": "<short phrase from the row's original data, or why
```

```
    hallucinated>}]]]}
Return the SAME number of rows as the input, same row_index values. No prose
outside the JSON.

== BATCH ==
{"analytical_goal": ..., "source_columns": [...],
 "rows": [{"row_index": N, "source_row": {...},
           "augmented_values": [{"column": ..., "value": ...}]]]}
```